APLICACIONES WEB II · DAM-2306 · UNIDAD 3 · PRÁCTICA 5

Cómo montar el balanceo en su proyecto

Con Laravel (`php artisan serve`), con Node o JavaScript, y con Docker

1. Antes de empezar: qué cuenta como nodo

La práctica pide tres nodos: **uno de balanceador y dos con su aplicación**. Cada nodo necesita su propia dirección y su propio proceso. Hay tres maneras válidas:

OPCIÓN	CÓMO QUEDA
Tres computadoras	En la misma red. Cada una con su IP (`ipconfig` para verla).
Máquinas virtuales	VirtualBox o similar, con red en modo puente para que cada una tenga su IP.
Contenedores Docker	Cada contenedor recibe su propia IP en la red de Docker. Ver la sección 5.

> **Dos terminales en una sola laptop** (como lo hicimos en clase con A y B) **sirven para armar y ensayar** la configuración. Para la entrega, cada nodo debe tener su propia dirección: tres computadoras, máquinas virtuales o Docker. Lo que elijan lo explican en el informe; no baja la calificación.

2. Lo que es igual para todos

1. **La misma versión de la aplicación en los dos nodos.** Si cambian algo, lo cambian en los dos.
2. **Una sola base de datos**, compartida por los dos nodos. Si cada nodo tiene su propia base, un usuario guarda algo en un nodo y no lo ve en el otro.
3. **Que cada nodo se identifique**: una cabecera propia o una marca visible, para comprobar quién atendió.
4. **Una ruta `/salud` que revise algo real** (que la base de datos conteste), no que siempre responda 200.
5. **La sesión debe sobrevivir al cambio de nodo**: afinidad, almacén compartido o testigo firmado, como vimos en clase.

EL BALANCEADOR · PLANTILLA DE NGINX

```
# Cambien las direcciones por las de sus nodos.
upstream proyecto {
    # least_conn;   ip_hash;   ← prueben al menos dos algoritmos
    server 192.168.1.20:8000 max_fails=2 fail_timeout=10s;
    server 192.168.1.21:8000 max_fails=2 fail_timeout=10s;
}

server {
    listen 8080;

    # El navegador pide el icono en cada recarga y se come un turno.
    location = /favicon.ico { return 204; }

    location / {
        proxy_pass http://proyecto;

        proxy_set_header Host              $http_host;   # con el puerto
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_next_upstream error timeout http_502 http_503 http_504;

        # Para comprobar el reparto: en F12 › Red se ve qué nodo atendió.
        add_header X-Nodo $upstream_addr always;
    }
}
```

> **Usen** `$http_host` **, no** `$host` **.** `$host` le quita el puerto: su aplicación arma los enlaces y las redirecciones hacia `http://localhost/...` sin el `:8080` , y el inicio de sesión los manda a una página que no existe. Lo probamos con una app Laravel y pasa exactamente eso.

LEVANTAR Y RECARGAR NGINX EN WINDOWS (SIN LARAGON)

Hagan una carpeta `nginx` con `conf\nginx.conf` , y las carpetas vacías `logs` y `temp` . El `nginx.conf` lleva:

```
worker_processes 1;
events { worker_connections 1024; }
http {
  include balanceo.conf;      # la plantilla de arriba, guardada en conf\balanceo.conf
}
```

```
# Dentro de la carpeta nginx. Levantar (la consola se queda quieta):
C:\laragon\bin\nginx\nginx-1.22.0\nginx.exe -p ./ -c conf/nginx.conf

# En otra consola, en la misma carpeta, cada vez que cambien el .conf:
C:\laragon\bin\nginx\nginx-1.22.0\nginx.exe -p ./ -c conf/nginx.conf -s reload

# Apagar:
C:\laragon\bin\nginx\nginx-1.22.0\nginx.exe -p ./ -c conf/nginx.conf -s stop
```

Si no tienen Laragon, descarguen Nginx para Windows de `nginx.org/en/download.html` y usen su `nginx.exe`. En Linux: `sudo apt install nginx`, la plantilla va en `/etc/nginx/conf.d/balanceo.conf` y se recarga con `sudo nginx -s reload`.

3. Laravel con `php artisan serve`

Sí se puede

Lo probamos en clase con una aplicación Laravel: dos `artisan serve` detrás de Nginx reparten bien y el inicio de sesión funciona.

ANTES DE LEVANTAR

```
composer install
npm run build          # no usen npm run dev detrás del balanceador
php artisan migrate
```

PARA ENSAYAR EN UNA LAPTOP · UNA TERMINAL POR NODO

```
php artisan serve --port=8001
php artisan serve --port=8002
```

Y en el `upstream`: `server 127.0.0.1:8001;` y `server 127.0.0.1:8002;`. Tarda unos segundos en arrancar; si al principio ven **502**, esperen y recarguen.

EN COMPUTADORAS DISTINTAS · EN CADA NODO

```
php artisan serve --host=0.0.0.0 --port=8000
```

`--host=0.0.0.0` es obligatorio: sin eso solo atiende a la misma computadora y el balanceador no lo alcanza. Si no conecta, revisen el firewall de Windows.

EL .ENV DE LOS DOS NODOS

```
APP_KEY=...            # LA MISMA en los dos nodos: copien el .env, no generen otra
APP_URL=http://IP-DEL-BALANCEADOR:8080
DB_HOST=IP-DE-LA-MAQUINA-CON-MYSQL
SESSION_DRIVER=database   # sesión en la base compartida: sobrevive al cambio de nodo
NODO=1                 # 1 en un nodo, 2 en el otro
```

> Con `SESSION_DRIVER=file`, cada computadora guarda la sesión en su disco: al cambiar de nodo, el usuario aparece sin sesión. Y si las `APP_KEY` son distintas, ningún nodo puede leer la cookie del otro.

CONFIAR EN EL BALANCEADOR · BOOTSTRAP/APP.PHP (LARAVEL 11 EN ADELANTE)

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->trustProxies(at: '*');   # mejor: at: 'IP-DEL-BALANCEADOR'
})
```

Sin esto, `$request->ip()` devuelve la IP del balanceador y no la del usuario. En Laravel 10 es la propiedad `$proxies` de `app/Http/Middleware/TrustProxies.php`.

SALUD E IDENTIFICACIÓN · ROUTES/WEB.PHP

```
use Illuminate\Support\Facades\DB;

Route::get('/salud', function () {
    DB::select('select 1');              # si la base no contesta, responde 500
    return response()->json([
        'nodo' => env('NODO', gethostname()),
        'ip_cliente' => request()->ip(),
    ]);
});
```

> **Para las mediciones:** `php artisan serve` es un servidor de desarrollo que en Windows atiende una petición a la vez. Sus números con Apache Bench van a salir bajos. Es válido para la práctica: lo que importa es comparar los escenarios en las mismas condiciones y comentarlo en el informe.

4. Node o JavaScript

CON SERVIDOR PROPIO (EXPRESS U OTRO)

```
const app = express();
app.set('trust proxy', true);           # req.ip con la IP real del usuario

const NODO = process.env.NODO || 'sin-nombre';
app.use((req, res, next) => { res.set('X-Servidor', NODO); next(); });

app.get('/salud', async (req, res) => {
  try {
    await db.query('SELECT 1');         # o lo que use su base de datos
    res.json({ nodo: NODO, bd: 'ok' });
  } catch (e) {
    res.status(503).json({ nodo: NODO, bd: 'caida' });
  }
});

app.listen(process.env.PORT || 3000, '0.0.0.0');
```

```
# Para ensayar en una laptop (PowerShell), una terminal por nodo:
$env:PORT=3001; $env:NODO="1"; node app.js
$env:PORT=3002; $env:NODO="2"; node app.js
```

> **La sesión:** `express-session` guarda las sesiones en memoria por defecto, y cada nodo tiene la suya. Usen un almacén compartido (por ejemplo, la sesión en su base de datos o en Redis) o un testigo firmado (JWT) que cualquier nodo pueda validar.

SOLO HTML, CSS Y JAVASCRIPT (SIN SERVIDOR PROPIO)

```
npx http-server -p 3001
npx http-server -p 3002
```

Una terminal por nodo, en la carpeta del sitio. Para saber quién atendió, usen la cabecera `X-Nodo` que agrega la plantilla del balanceador. Si su proyecto no tiene servidor, la parte de sesión la platicamos en clase para ver cómo la cubren.

5. Con Docker

Necesitan Docker Desktop (en Windows usa WSL2). Con un solo archivo levantan los tres nodos y la base de datos, cada contenedor con su propia IP.

DOCKERFILE · SI SU PROYECTO ES NODE

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "app.js"]
```

DOCKERFILE · SI SU PROYECTO ES LARAVEL

```
FROM php:8.3-cli
RUN docker-php-ext-install pdo_mysql
WORKDIR /app
COPY . .
CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

En Laravel, corran `composer install` y `npm run build` antes de construir la imagen.

DOCKER-COMPOSE.YML

```
services:
  app1:
    build: .
    environment:
      NODO: "1"
      DB_HOST: db
      DB_DATABASE: proyecto
      DB_USERNAME: root
      DB_PASSWORD: secreto
  app2:
    build: .
    environment:
      NODO: "2"                # lo único que cambia entre los dos
      DB_HOST: db
      DB_DATABASE: proyecto
      DB_USERNAME: root
      DB_PASSWORD: secreto
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: proyecto
      MYSQL_ROOT_PASSWORD: secreto
  balanceador:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
    volumes:
      - ./balanceo.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [app1, app2]
```

En `balanceo.conf` usen la plantilla de la sección 2, con dos cambios: `listen 80;` y los nodos por su nombre de servicio:

```
upstream proyecto {
    server app1:8000 max_fails=2 fail_timeout=10s;   # 3000 si es Node
    server app2:8000 max_fails=2 fail_timeout=10s;
}
```

LOS COMANDOS QUE VAN A USAR

```
docker compose up -d --build                    # levantar todo
docker compose exec app1 php artisan migrate    # solo Laravel, una vez
docker compose exec balanceador nginx -s reload # después de editar balanceo.conf
docker compose stop app1                        # tirar un nodo con la prueba
corriendo
docker compose start app1                       # reincorporarlo
docker compose logs -f balanceador              # ver qué pasa
docker compose down                             # apagar todo
```

Si al recargar Nginx no toma el cambio del `.conf`, usen `docker compose restart balanceador`. El sitio queda en `http://localhost:8080`. En el informe, el diagrama lleva los nombres de los contenedores, sus IP (`docker network inspect`) y los puertos.

6. Medir con Apache Bench

```
# Un servidor solo, sin balanceador:
ab -n 1000 -c 20 http://IP-DE-UN-NODO:8000/

# El conjunto completo, a través del balanceador (mismas -n y -c):
ab -n 1000 -c 20 http://IP-DEL-BALANCEADOR:8080/
```

Con Laragon, `ab.exe` está en `C:\laragon\bin\apache\httpd-...\bin\`. Usen la IP o `127.0.0.1`, no `localhost`, y no olviden la diagonal final. Del resultado registren **Requests per second** y los percentiles **50 %, 90 % y 99 %**. Repítanlo con el otro algoritmo y, en otra corrida, apaguen un nodo a la mitad.